# 2 سامي عرفات أحمد المعايطة

SJF و SRTF

### 📌 شو إلك بالمشروع

- **Phase 2 · فيديو 19 نوفمبر:** خوارزمية SJF
- **Phase 3 · 17 ديسمبر:** خوارزمية SRTF (الـ preemption)

الكود كله بملف **CPU_SchedLab.py**. افتحه بـ VS Code واكبس **Ctrl + G** واكتب رقم السطر اللي بالصورة.

## Phase 2 · فيديو 19 نوفمبر خوارزمية SJF

### خوارزمية SJF (F4)

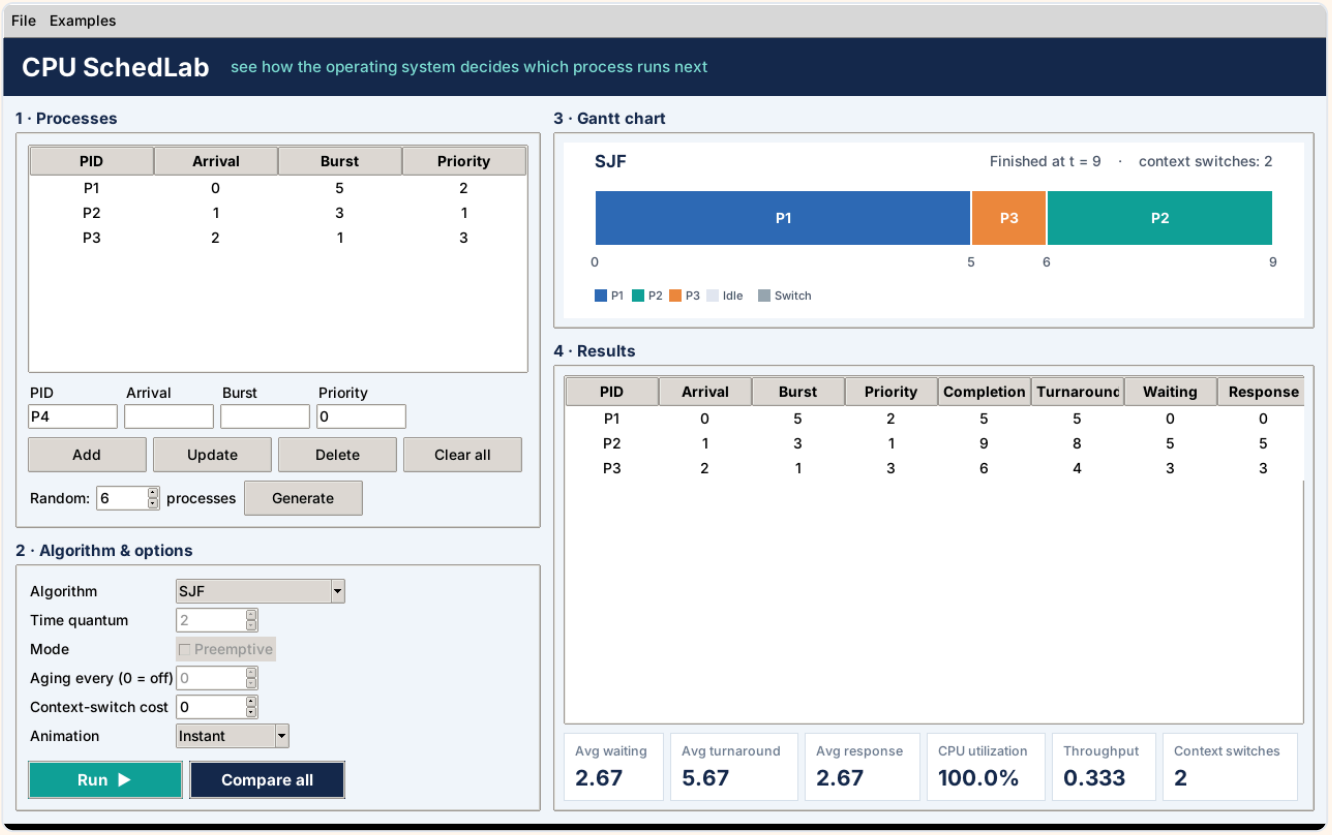
CPU_SchedLab.py — lines 121–138

```
121  def sjf_schedule(processes, options):
122      """F4 SJF: when the CPU is free, run the ARRIVED process with the SHORTEST burst."""
123      cpu = Dispatcher(options.switch_cost)
124      time = 0
125      not_started = list(processes)
126      while not_started:
127          ready = [p for p in not_started if p.arrival <= time]
128          if not ready:                                   # nobody ready -> idle
129              next_arrival = min(p.arrival for p in not_started)
130              cpu.idle(time, next_arrival)
131              time = next_arrival
132              continue
133          p = min(ready, key=lambda p: (p.burst, p.arrival))  # shortest burst wins
134          time = cpu.dispatch(p.pid, time)
135          cpu.run(p.pid, time, time + p.burst)            # runs to the end
136          time += p.burst
137          not_started.remove(p)
138      return cpu.result("SJF", processes)
```

### 🎤 شو بتحكي للدكتورة

بخوارزمية **SJF**، كل ما يفضى المعالج، بختار من الـ processes اللي وصلت فعلاً الـ process اللي الـ burst تبعها **أقصر** اشي. الـ **ready** هي الـ processes اللي وصلت ولسّا ما اشتغلت. إذا ما في ولا وحدة جاهزة، المعالج بضل idle لحد أقرب وحدة بتوصل. الاختيار بيصير بالدالة **min** على (burst, arrival)، يعني الأقصر أول، وإذا تساووا بناخد اللي وصلت أول. وبعد ما نختارها بتضلها شغّالة لحد ما تخلّص، لأنه SJF هي **non-preemptive**. بمثالنا، عند الزمن 5 كانت P2 بدها 3 و P3 بدها 1، فاخترنا P3، وصار معدّل الانتظار **2.67** بدل 3.33 تبع FCFS. SJF بتعطي **أقل معدّل انتظار ممكن**، بس مشكلتها إنه بالواقع ما بنعرف طول الـ burst من قبل، فالنظام بيتوقّعه باستخدام **exponential averaging**.

## 🖥️ شو بتعرض عالشاشة




بختار SJF وبكبس **Run**: الترتيب P1 بعدين P3 بعدين P2، ومعدّل الانتظار **2.67**، أحسن من FCFS اللي كان 3.33.

## 💬 إذا سألتك الدكتورة

### ❓ ليش ما اشتغلت P3 من أول ما وصلت؟

لأنه SJF ما بتقاطع؛ P1 كانت شغّالة فضلّت لحد 5.

### ❓ ليش SJF هي الأحسن بمعدّل الانتظار؟

لأنه لما القصيرة تشتغل أول، عدد أقل من الـ processes بيستنّى وقت طويل.

### ❓ شو مشكلتها؟

ما بنعرف طول الـ burst مسبقاً، وممكن الطويلة يصيرلها starvation.

## خوارزمية SRTF (F5)

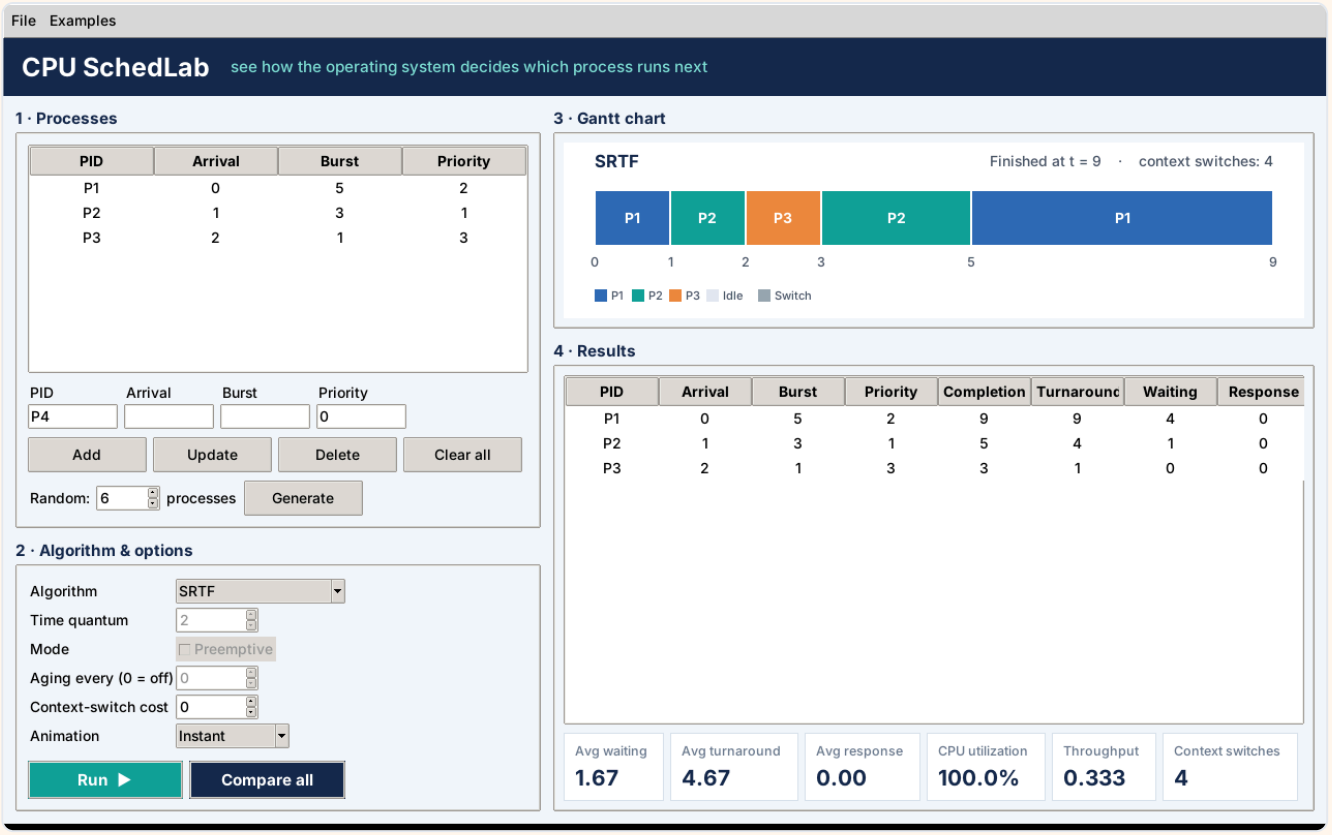
CPU_SchedLab.py lines 141–160

```python
def srtf_schedule(processes, options):
    """F5 SRTF: every time unit, run the process with the smallest REMAINING time."""
    cpu = Dispatcher(options.switch_cost)
    remaining = {p.pid: p.burst for p in processes}
    time, running = 0, None
    while any(remaining.values()):
        ready = [p for p in processes if p.arrival <= time and remaining[p.pid] > 0]
        if not ready:
            next_arrival = min(p.arrival for p in processes if remaining[p.pid] > 0)
            cpu.idle(time, next_arrival)
            time = next_arrival
            continue
        # smallest remaining time; on a tie keep the running one (no useless switch)
        p = min(ready, key=lambda p: (remaining[p.pid], p.pid != running, p.arrival))
        time = cpu.dispatch(p.pid, time)
        cpu.run(p.pid, time, time + 1)                    # ONE unit, then decide again
        time += 1
        remaining[p.pid] -= 1
        running = p.pid
    return cpu.result("SRTF", processes)
```

### 🎤 شو بتحكي للدكتورة

**SRTF** هي النسخة اللي بتقاطع من SJF. بخلّي معي قاموس اسمه **remaining** فيه قدّيش ضايل لكل process. وبكل وحدة زمن برجع بقرّر من جديد: بختار الـ process اللي ضايلها أقل وقت، وإذا تساوت مع اللي شغّالة هسا بخلّي اللي شغّالة، عشان ما يصير context switch على الفاضي. بعدين بشغّلها **وحدة زمن وحدة بس**، وبرجع بقرّر، وهاد بالزبط اللي بيخلّيها **preemptive**. بالمثال: عند الزمن 1 وصلت P2 وبدها 3، وهاد أقل من الـ 4 اللي ضايلين لـ P1، فقاطعتها. وعند الزمن 2 وصلت P3 وبدها 1، فقاطعت P2. النتيجة معدّل انتظار **1.67** وهو الأحسن، بس كان الثمن 4 context switches، وممكن الـ processes الطويلة يصيرلها **starvation**.

### 🖥️ شو بتعرض عالشاشة




بختار SRTF وبكبس **Run**: المقاطعات بتبيّن بالرسم: P1 بعدين P2 بعدين P3 بعدين P2 بعدين P1، وعدد الـ context switches أربعة، ومعدّل الانتظار **1.67**.